esprit
Se former autrement

LANGAGE C#

Cours présenté par Tarek Ayari

TYPES DE DONNÉES

Type	Classe	Description	Exemples
bool	System.Bool	Booléen (vrai ou faux : true ou false)	true
char	System.Char	Caractère Unicode (16 bits).	'A' 'λ' 'ω'
sbyte	System.SByte	Entier signé sur 8 bits (1 octet)	-128
byte	System.Byte	Entier non signé sur 8 bits (1 octet)	255
short	System.Int16	Entier signé sur 16 bits	-129
ushort	System.UInt16	Entier non signé sur 16 bits	1450
int	System.Int32	Entier signé sur 32 bits	-100000
uint	System.UInt32	Entier non signé sur 32 bits	8000000

TYPES DE DONNÉES

Type	Classe	Description	Exemples
long	System.Int64	Entier signé sur 64 bits	-2565018947302L
ulong	System.UInt64	Entier non signé sur 64 bits	8000000000000L
float	System.Single	Nombre à virgule flottante sur 32 bits	3.14F
double	System.Double	Nombre à virgule flottante sur 64 bits	3.14159
decimal	System.Decimal	Nombre à virgule flottante sur 128 bits	3.1415926M
string	System.String	Chaîne de caractères unicode	"C#"
object	System.Object	Type Objet	

CONVERSIONS DE DONNÉES

Le cast

Le cast est une action qui permet de convertir une donnée d'un type vers un autre type.

- **Le cast implicite**

Le cast implicite est un cast qui ne pose aucun problème, car il n'entraine pas de perte de données. Par exemple si on veut convertir un type «int» en un type «long», la conversion ne posera jamais problème car le type «long» englobe le type «int».

```
static void Main(string[] args)
{
        int age = 42;
        long longAge = age; //Ici on convertit notre int en long

        Console.WriteLine("Mon âge est : {0}",longAge);
        Console.ReadLine();
}
```

CONVERSIONS DE DONNÉES

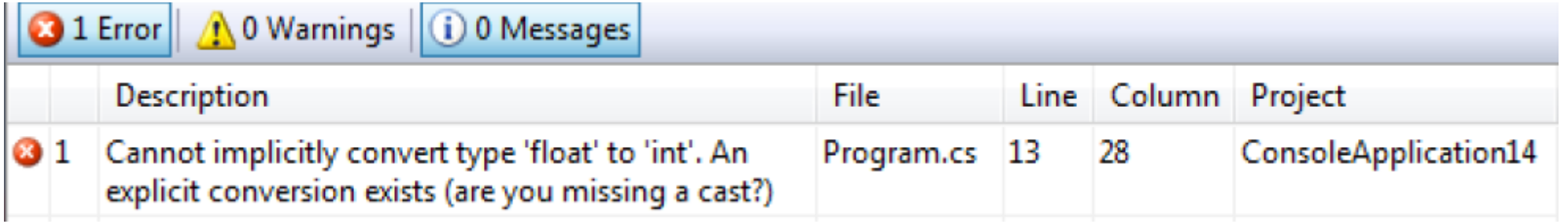
- **Le cast explicite**

Tout d'abord nous allons essayer le code suivant et observer ce qu'il se passe.

```
static void Main(string[] args)
{
        float resultatPrecis = 17.83f;
        int resultat = resultatPrecis;

        Console.WriteLine("Le résultat est : {0}", resultat);
        Console.ReadLine();
}
```

Comme vous pouvez l'imaginer, Visual Studio ne va pas compiler et nous renvoie un message d'erreur



CONVERSIONS DE DONNÉES

Pour éviter ce message d'erreur et réussir à la compilation, il va nous falloir définir notre conversion de manière explicite.

```
static void Main(string[] args)
{
      float resultatPrecis = 17.83f;
      int resultat = (int)resultatPrecis; //Conversion explicite.

      Console.WriteLine("Le résultat est : {0}", resultat);
      Console.ReadLine();
}
```

```
Le résultat est : 17
```

On observe que notre conversion vers les entiers a bien supprimé les chiffres après la virgule.

CONVERSIONS DE DONNÉES

Conversion d'une chaine en nombre

Pour convertir une chaîne en nombre on va utiliser la fonction **« Parse »** précédée du type voulu.

Par exemple si l'on veut convertir « x » en entier on va faire «int.Parse(x)» si on le veut en décimal on écrira «float.Parse(x)»

ou en « long » « long.Parse(x) » etc.

```
static void Main(string[] args)
{
      //Deux chaines de caractères :
      string maChaineA = "42";
      string maChaineB = "23";

      //Conversion en entiers puis additon :
      int addition = int.Parse(maChaineA) + int.Parse(maChaineB);

      //Affichage et pause.
      Console.WriteLine(addition);
      Console.ReadLine();

}
```

CONVERSIONS DE DONNÉES

Conversion d'un nombre en chaine

La conversion d'un nombre vers une chaine de caractère s'effectue grâce à la fonction **« ToString()»** que l'on va placer à

la suite de notre variable à convertir. Pour convertir notre nombre « x » on écrira donc « x.ToString() ».

```
static void Main(string[] args)
{
      int nombre = 256;

      //Conversion de nombre en chaîne:
      string ChaineNombre = nombre.ToString();

      //Affichage et Pause:
      Console.WriteLine(ChaineNombre);
      Console.ReadLine();
}
```

CONVERSIONS DE DONNÉES

La classe Convert

La classe Convert contient un grand nombre de méthodes de conversion de types. Microsoft indique que cette classe :

"constitue une façon, indépendante du langage, d'effectuer les conversions et est disponible pour tous les langages qui ciblent le Common Language Runtime.

```
using System;
public class Demo {
   public static void Main() {
      decimal decimalVal;
      string stringVal = "2,345.26";
      decimalVal = System.Convert.ToDecimal(stringVal);
      System.Console.WriteLine("String converted to decimal = {0} ", decimalVal);
   }
}
```

```
String converted to decimal = 2345.26
```

STRUCTURE CONDITIONNELLE

If Syntaxe:

```
if ( condition )
{
        // instructions à réaliser si CONDITION vaut true
} else {
        // instructions à réaliser si CONDITION vaut false
}
```

Switch Syntaxe:

```
switch(nom_de_la_variable)
{
    case valeur_1:
            Instructions à exécuter dans le cas où la variable vaut valeur_1
        break;
    case valeur_2:
            Instructions à exécuter dans le cas où la variable vaut valeur_2
        break;
    default:
            Instructions à exécuter dans le cas où la variable vaut une valeur
autre que valeur_1 et valeur_2
        break;
}
```

STRUCTURES ITERATIVES

For Syntaxe:

```
for ( initialisation; condition; instructionDIteration )
{
        // bloc d'instructions à réaliser tant que condition
vaut true
}
```

Foreach Syntaxe:

```
 foreach (type Name1 in ListName)
 {
      …
 }
```

While Syntaxe:

```
while ( condition )
{
        // bloc d'instructions à réaliser tant que condition
vaut true
}
```

Do While Syntaxe:

```
 do
     {
      …
     }
    while (logical condition);
```

LES FONCTIONS

- Une fonction est une portion de code qui effectue une tâche ou un calcul.
- Une fonction est désignée par un identificateur qui permet de l'appeler à partir d'autres endroits du programme
- La fonction peut retourner une valeur, c'est-à-dire renvoyer un résultat.
- Une fonction peut prendre des arguments ou paramètres qui désignent les données d'entrée sur lesquelles travaille la fonction.

La déclaration d'une fonction se fait en 2 parties :

- On commence par donner son prototype qui contient toutes les informations d'identification de la fonction: son nom, ses paramètres, son type de retour…
- Puis on donne son corps entre accolades qui contient les instructions de la fonction.

LES FONCTIONS

Syntaxe:

```
TypeDeRetour IdentificateurFonction(TypeParametre1 identificateurParametre1, TypeParametre2 identificateurParametre2, ...)
// prototype
{      // début du corps
       // intructions de la fonction
       return valeurDeRetour;
}      // fin du corps
```

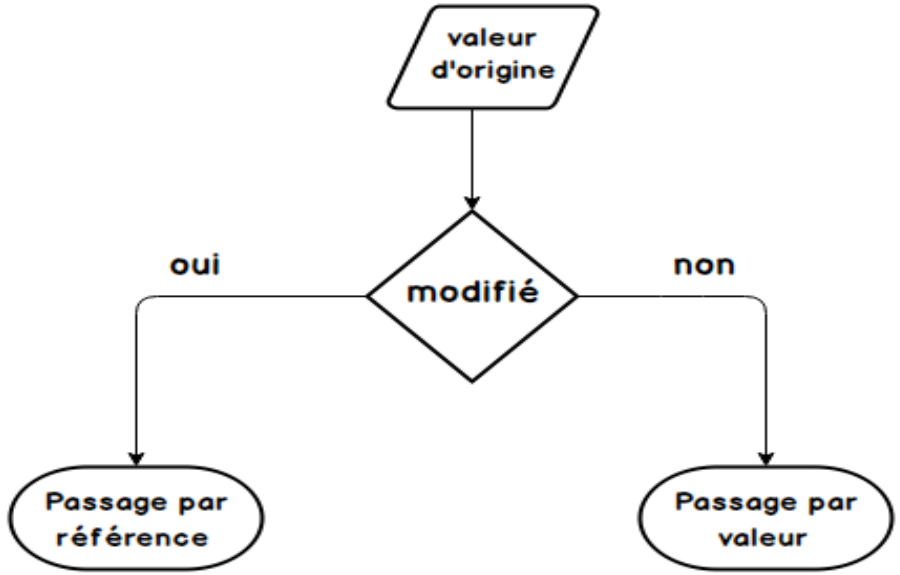
```
int Fonction2(int a)
{
        a += 1;
        return a;
}
```

```
void Fonction1()
{
        int b = 7;
        b = Fonction2(b); // Fonction2(b) est une expression de type int et de valeur 8
}
```

PASSAGE PAR RÉFÉRENCE ET PASSAGE PAR VALEUR

Il existe deux manières de transmettre des arguments ou des paramètres aux fonctions :

- **Passage par valeur**
- **Passage par référence**



	Passage par valeur	Passage par référence
Variable	Une copie de la variable est transmise.	La variable elle-même est transmise.
L'Effet	La modification du copie de variable ne modifie pas la valeur d'origine de la variable en dehors de la fonction.	La modification de la variable affecte également la valeur de la variable en dehors de la fonction.

PASSAGE PAR VALEUR

```
static int F(int n)
{
   ++n; // modifie le n local à F
   return n;
}

static void G()
{
   int n = 3;
   int m = F(n); // passage par valeur
   // Après l'appel, m : 4 et n : 3
   Console.Write($"Après l'appel, m : {m} et n : {n}");
}
```

PASSAGE PAR REFRENCE

C# exige que le code client appose **ref** au point d'appel dans un tel cas.

```
static void Permuter(ref int a, ref int b)
{
   int temp = a;
   a = b;
   b = temp;
}
static void Main()
{
   int m = 2;
   int n = 3;
   // Avant l'appel, m : 2 et n : 3
   Console.Write($"Avant l'appel, m : {m} et n : {n}");
   Permuter(ref m, ref n);
   // Après l'appel, m : 3 et n : 2
   Console.Write($"Après l'appel, m : {m} et n : {n}");
}
```

LES TABLEAUX

- Etant donné un type T, la notation T[] désigne le nouveau type tableau d'éléments de type T.
- Par exemple int[] désigne le type tableau d'entiers qui est différent du type entier.

- Tableaux unidimentionnels:

```
TypeDesElements [ ] monTableau1; // déclaration d'un tableau
TypeDesElements [ ] monTableau2 = new TypeDesElements [tailleDuTableau]; // déclaration et création d'un tableau
int [ ] monTableau3 = {1, 5, 6, 7}; // déclaration d'un tableau et initialisation explicite
```

- Tableaux multidimensionnels :

```
TypeDesElements [,] monTableau1; // déclaration d'un tableau bidimentionnel
TypeDesElements [,,] monTableau2; // déclaration d'un tableau tridimentionnel
TypeDesElements [,] monTableau3 = new TypeDesElements [nombreDeLignes, nombreDeColonnes] ; // déclaration et initialisation d'un tableau
int [,] monTableau4 = { {1, 5, 6}, {2, 9, 7} }; // déclaration d'un tableau 2d et initialisation
```

ACCÈS AUX ÉLÉMENTS D'UN TABLEAU

- Les cases d'un tableau sont numérotées de 0 à la taille du tableau moins un.
- Le numéro d'une case est appelé son indice et permet d'accéder à cette case.

Syntaxe

```
int [] monTableau1 = {1, 2, 3};

monTableau1[0] = 4;  //modification de la valeur de la case d'indice 0: monTableau1 = {4, 2, 3}

int a = monTableau1[2]; // lecture de la valeur de la dernière case du tableau: a = 3

int [,] monTableau2 = { {1, 2, 3},    {4, 5, 6}};

monTableau2[0, 0] = 7;  // monTableau2 = { {7, 2, 3}, {4, 5, 6}}

int b = monTableau2[1, 2];  // b = 6  (deuxième ligne, troisième colonne)
```

TAILLE D'UN TABLEAU

Pour connaitre la taille d'un tableau unidimensionnel on utilise la propriété Length:

```
int [] monTableau1 = {1, 2, 3};

int taille = monTableau1.Length; // taille = 3
```

Pour connaitre la taille d'un tableau bidimensionnel on utilise la méthode GetLength(dimension):

```
int [,] monTableau1 = { {1, 2, 3}, {4, 5, 6} };

int lignes = monTableau1.GetLength(0);   // 2

int colonnes = monTableau1.GetLength(1); // 3
```

PARCOURS DES TABLEAUX UNIDIMENSIONNEL

Avec une boucle for ou une boucle foreach (si les indices des éléments ne nous intéressent pas).

```
bool [] monTableau = {true, true, false};

for (int i = 0; i < monTableau.Length; i++)
{
    ... monTableau[i] ...
}

foreach (bool valeur in monTableau)  // se lit: "pour toute valeur de type bool dans monTableau"
{
    ... valeur ...
}
```

PARCOURS DES TABLEAUX BIDIMENSIONNEL

```
// un tableau à 2 dimensions initialisé
 double[,] réels = new double[,] { { 0.5, 1.7 }, { 8.4, -6 } };
for (int i = 0; i < réels.GetLength(0); i++) {
        for (int j = 0; j < réels.GetLength(1); j++) {
              Console.Out.WriteLine ("réels[{0},{1}]={2}", i, j, réels[i, j]);
        }//for j
 }//for i
```

EXERCICE 1

Exercice



Document
Microsoft Word

Correction



Document
Microsoft Word

LES CLASSES

Une classe est un regroupement de données, et des méthodes pour initialiser ces données et les manipuler.

```
class nomDeLaClasse
{
    // Déclaration des champs

    // Déclaration des méthodes
}
```

LES CLASSES: CONSTRUCTEUR/DESTRUCTEUR

Le constructeur: Le constructeur d'une classe est appelé à chaque fois que vous voulez créer un objet à partir de cette classe.

- Lorsqu'il est appelé, le constructeur réserve un emplacement mémoire pour votre objet
- Vous n'êtes pas obligés d'écrire vous-mêmes le code du constructeur ; dans ce cas, un constructeur "par défaut" est utilisé.

Le destructeur: est une méthode appelée lors de la destruction d'un objet. Son nom est celui de la classe, précédé d'un tilde '~'.

- Rien ne vous oblige à mettre un destructeur. C'est seulement si vous voulez faire quelque chose de particulier à sa destruction. Cela peut notamment servir à libérer de la mémoire et à bien gérer les ressources

LES CLASSES: CONSTRUCTEUR/DESTRUCTEUR

```
public class Person
{
// Constructeur
    public Person()
        {
            Console.WriteLine("Nouvelle personne créée.");
        }
//Destructeur
    ~Person()
        {
            Console.WriteLine("Objet détruit.");
        }
}
```

CRÉATION D'UN OBJET

La création d'un objet, ou instanciation, est réalisé avec l'opérateur new qui retourne une référence vers le nouvel objet créé.

```
identificateurVariable = new IdentificateurClasse(parametre1, parametre2, ...);
```

Exemple:

```
/* déclaration d'une variable de type Bitmap*/
Bitmap image = new Bitmap(400,200);
/* déclaration d'une variable de type Voiture*/
Voiture voiture = new Voiture();
```

L'instance courante et est désignée par le mot clé this

LES INTERFACES

Une interface est un modèle de classe. Elle contient des méthodes et des propriétés, avec la particularité qu'elle ne fournit pas l'implémentation des méthodes. Cela veut dire qu'on décrit juste les méthodes sans décrire ce qu'elles font..

```
interface IAnimal
{
    string Name { get; }
    void Move();
}
```

LES INTERFACES

Toute classe implémentant une interface doit implémenter ses méthodes (avec les mêmes signatures) et ses propriétés.

```
class Dog : IAnimal
{
      private string m_name;
      // On implémente la propriété Name accessible en lecture.
      public string Name
      {
            get { return m_name; }
      }
    // On implémente la méthode Move.
      public void Move()
      {
            Console.WriteLine("{0} bouge.", m_name);
      }
}
```

LES INTERFACES

Utiliser plusieurs interfaces: Suivant vos besoins, vous pouvez même utiliser plusieurs interfaces. Pour cela, il faut séparer les interfaces par des virgules

```
class SampleClass : IControl, ISurface
{
    // Code de la classe
}
```

CLASSES ET MÉTHODES ABSTRAITES

- Une classe abstraite possède au moins une méthode abstraite, c'est à dire ne comportant aucune implémentation (pas de code).
- Une telle classe ne peut être instanciée avec l'opérateur new. Il faut utiliser une sous-classe implémentant les méthodes abstraites de la classe de base.
- Les classes abstraites servent à factoriser du code, tandis que les interfaces servent à définir des contrats de service.

CLASSES ET MÉTHODES ABSTRAITES

```
abstract class Shape
{
        protected string nom ;
        public abstract void Draw() ;   // méthode abstraite : pas de block {}
}

class Triangle : Shape
{
        protected Point p1, p2, p3 ;
        public override void Draw()     { ... } // (re)définition de la méthode Draw
}
```

Une classe abstraite ne peut pas être instanciée (elle est incomplète car les méthodes abstraites n'ont pas de corps)

LE TYPE ENUM

Les énumérations en C# permettent de définir un ensemble de constante qui sont liées entre elles.
Création d une énumération des jours de la semaine:

```
enum WeekDays
{
    Monday,
    Tuesday,
    Wednesday,
    Thursday,
    Friday,
    Saturday,
    Sunday
}
```

LE TYPE ENUM

Valeurs énumérées: Si les valeurs ne sont pas attribuées aux enum membres, le compilateur attribuera des valeurs entières à chaque membre en commençant par zéro par défaut.

```
enum WeekDays
{
    Monday,     // 0
    Tuesday,    // 1
    Wednesday,  // 2
    Thursday,   // 3
    Friday,     // 4
    Saturday,   // 5
    Sunday      // 6
}
```

LE TYPE ENUM

Vous pouvez même attribuer des valeurs différentes à chaque membre..

```
enum Categories
{
    Electronics = 1,
    Food = 5,
    Automotive = 6,
    Arts = 10,
    BeautyCare = 11,
    Fashion = 15,
    WomanFashion = 15
}
```

LE TYPE ENUM

Accéder à une énumération: Un enum est accessible en utilisant la syntaxe :

enum.member

```
Console.WriteLine(WeekDays.Monday.ToString()); // Monday
Console.WriteLine((int)WeekDays.Tuesday); // 1
```

EXERCICE 2

Exercice



Document
Microsoft Word

Correction



Document
Microsoft Word

ENCAPSULATION: VISIBILITÉ

- En programmation orientée objet, le principal mécanisme de l'encapsulation repose sur les qualificatifs de visibilité.
- Ces qualificatifs permettent de contrôler si une méthode ou un champ peut être utilisé par une autre classe.
- Nous avons déjà vu le qualitificatif public qui signifie qu'un membre est accessible sans restriction à tout le monde (il fait parti de l'interface de l'objet).
- L'opposé du qualificatif public est le qualitificatif private qui signifie qu'un membre n'est accessible que depuis la classe courante, donc caché au monde extérieur.
- Essayer d'accéder à un membre privé depuis une autre classe génère une erreur de compilation.

ENCAPSULATION: EXEMPLE

```
class A{
        public int champPublic = 1;
        private int  champPrive = 2;

        public void MethodePublique(){}
        private void MethodePrivee(){}
}

class B{
        public void Test()
        {
                A a = new A();
                a.champPublic++;        // ok car champPublic est public
                // a.champPrive++;      ERREUR car nous sommes dans la classe B
                a.MethodePublique();
                // a.MethodePrivee();   ERREUR car nous sommes dans la classe B

        }
}
```

ENCAPSULATION: VISIBILITÉ

- On trouvera la visibilité protégée fixée via le mot clé protected. accès fermé pour l'extérieur mais ouvert pour les classes dérivées
- La visibilité interne fixée par le mot clé internal permet aux membres d'être visibles seulement dans l'assembly (dll ou projet).
- Notez qu'il est possible d'avoir la combinaison entre la visibilité interne et protégée via le mot clé internal protected ou privée et protégée via le mot clé private protected.

```
public class ClassExample
{
    private int field;
    0 références
    protected ClassExample()
    {}
    0 références
    internal void M() { }
    0 références
    public int Property { get; set; }
    0 références
    internal protected void M2() { }
    0 références
    private protected void M3() { }
}
```

MEMBRES STATIQUES

Le qualificateur static permet de déclarer des membres de classe, aussi appelés membres statiques, qui ne dépendront pas d'une instance particulière de la classe. .

```
class A
{
        double valeur;
        public static int compteur = 0; // nombre
d'objets de type A créés

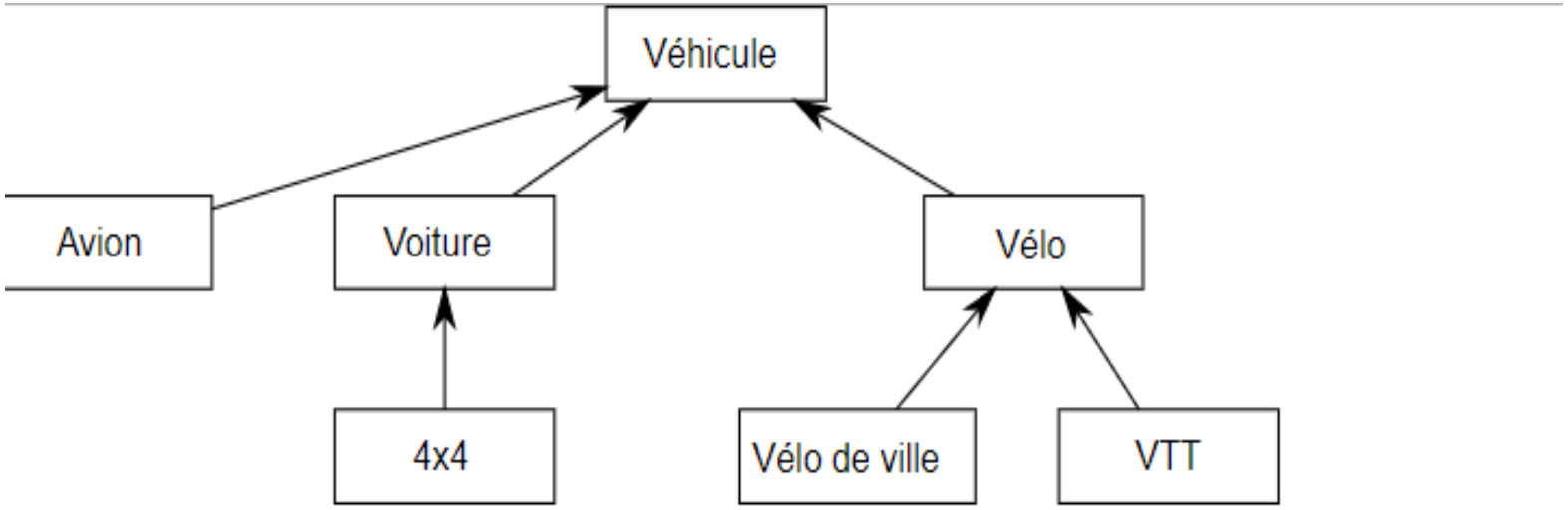
        public A(double valeur){
                this.valeur = valeur;
                A.compteur++;              // on vient de
créer un nouvel objet
        }
```

```
 public static void Test()       {  // une méthode statique
A a1 = new A(3.1);
WriteLine(A.compteur); // 1
A a2 = new A(3.2);
WriteLine(A.compteur); // 2
WriteLine(a1.valeur);  // 3.1 accès à un champ d'instance à
partir d'une référence
}
}
```

L'HÉRITAGE

L'héritage permet de construire une classe B à partir d'une classe existante A. Il s'agit d'une sorte d'héritage biologique. La classe B va ainsi hériter des variables, des méthodes, bref de tous les membres de A puis, on va ajouter des éléments supplémentaires propres à la classe B.

- Si B hérite de A, on dit que B est une classe fille, une sous-classe, une classe enfant ou encore une classe dérivée de A.
- A l'inverse, A se nomme la classe mère, la classe de base, la classe parent ou encore la super classe de B.



L'HERITAGE: PRINCIPE

Syntaxe

```
class IdentificateurClasse : IdentificateurClasseParent
{

}
```

- Une classe ne peut avoir qu'une seule classe parent. L'héritage multiple est interdit.

- Les cycles sont interdits : A ne peut hériter de B si B est un descendant de A.

Exemple:

```
class Vehicule
{
        public string nom;
        public void AfficheNom(){
                WriteLine(nom);
        }
}

class Voiture : Vehicule
{
        public int puissance;
        public void Info() {
                AfficheNom();  // on hérite des méthodes de Vehicule
                WriteLine(puissance);  // on hérite des champs de Vehicule
        }
}
```

L'HERITAGE: PRINCIPE

•Lorsque qu'une classe B hérite d'une classe A, B n'a pas accès aux membres privés de A. Afin d'avoir un contrôle plus fin sur la visibilité, l'héritage introduit le nouveau niveau de visibilité protected qui ouvre l'accès aux enfants mais pas à l'extérieur.

b1.MethodeA(); // OK car b1 est de type déclaré B qui hérite de A

b1.MethodeB(); // OK car b1 est de type déclaré B

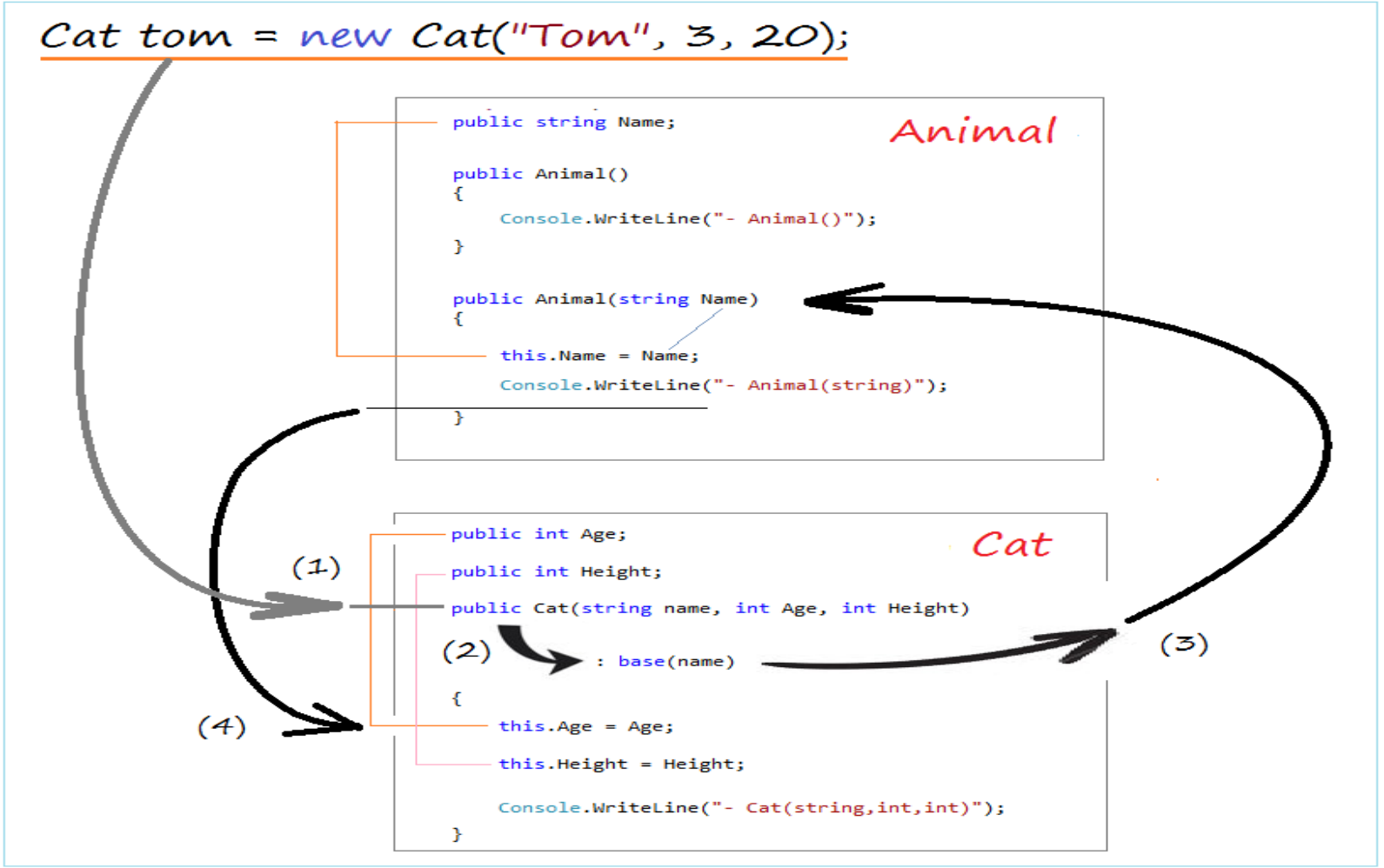
a2.MethodeA(); // OK car a2 est de type déclaré A

On se retouve ainsi avec 3 niveaux de visibilités possibles pour les membres de la classe A:

- **public :** accès ouvert pour l'extérieur et pour les classes dérivées
- **protected :** accès fermé pour l'extérieur mais ouvert pour les classes dérivées
- **private :** accès fermé pour l'extérieur et fermé pour les classes dérivées

L'HERITAGE: LE MOT CLE BASE

- Le mot-clé base permet d'accéder aux membres de la classe de base depuis une méthode d'une classe dérivée.



TYPES GÉNÉRIQUES

Il est possible en C# de spécifier un type dont on ne connaît pas l'implémentation à l'avance. Ce sont les types génériques. Pour spécifier des types génériques il suffit de mettre au niveau du type une liste de noms après la déclaration du type en question en utilisant les symboles < et > (<> est appelé diamant).

Voici un exemple avec une classe

```
class GenericExample<T1,T2>
{
    T1 field;
    0 références
    GenericExample(T1 fValue)
    {
        field = fValue;
    }
    0 références
    T2 Method()
    {
        return default(T2);
    }
}
```

TYPES DÉLÉGUÉ

C'est un sorte de pointeur vers une fonction. Pour déclarer un délégué on doit utiliser le mot clé delegate en spécifiant le nom de ce délégué, par exemple

```
public delegate int DelExample(string str);
```

Ici le délégué n'a pas de corps et il doit représenter la signature des fonctions (ou méthodes) qui peut les pointer.

Utiliser le type délégué si on veut **passer une fonction en paramètre**

TYPES DÉLÉGUÉ : UTILISATION

```
// "addnum" est un type delegue
public delegate void addnum(int a, int b);


 // method qui va etre utilise dans le type delegue
    public void sum(int a, int b)
    {
        Console.WriteLine("(100 + 40) = {0}", a + b);
    }
 // instantiating the delegates
    addnum del_obj1 = new addnum(obj.sum);
 // pass the values to the methods by delegate object
    del_obj1(100, 40);
```

TYPES DÉLÉGUÉ PREDFINI : ACTION / FUNC

- **Action et Func sont des délégués prédéfinis.**
- Les actions ne peuvent prendre que des paramètres d'entrée, tandis que Funcs peut prendre des paramètres d'entrée et de sortie. C'est la plus grande différence entre eux.
- Une action ne peut pas renvoyer de valeur, alors qu'une fonction doit renvoyer une valeur dans les autres mots

TYPES DÉLÉGUÉ PREDFINI : ACTION

- **Passage de fonction**

```
public static void RunActionFuncSample()
{
    Action<int, int> sumAndShowOnConsole = new Action<int, int>(Sum);
    sumAndShowOnConsole(10, 20);
}

public static void Sum(int number1, int number2)
{
    Console.WriteLine(number1 + number2);
}
```

TYPES DÉLÉGUÉ PREDFINI : ACTION

- **Action fléchées** : prend un argument à utiliser dans son corps

```
Action<int> showAgeAction = (age) =>
{
    Console.WriteLine($"The person is {age} years old");
};

showAgeAction(10);
```

TYPES DÉLÉGUÉ PREDFINI : FUNC

- **Passage de fonction**

```
public static void RunFuncSample()
{
    Func<int> generateRandomFunc = new Func<int>(GenerateRandomFunc);

    Console.WriteLine(generateRandomFunc());

    Func<int, int, string> sumFunc = new Func<int, int, string>(SumAsStringFunc);
    string sum = sumFunc(12, 24);
    Console.WriteLine(sum);
}

public static int GenerateRandomFunc()
{
    return Random.Shared.Next();
}

public static string SumAsStringFunc(int number1, int number2)
{
    return (number1 + number2).ToString();
}
```

TYPES DÉLÉGUÉ PREDFINI : FUNC

- **Fonctions fléchées** : prend un argument à utiliser dans son corps

```
Func<int, int, int> sumFunc = (number1, number2) =>
{
    return number1 + number2;
};

int sum = sumFunc(12, 36);

Console.WriteLine(sum);
```

POLYMORPHISME AD HOC: SURCHARGE

Le terme polymorphisme désigne la capacité d'une méthode à pouvoir prendre un comportement différent suivant le contexte: types des paramètres et type de l'objet courant.

Par exemple, si l'on veut créer une fonction calculant le maximum de deux nombres, il serait pratique qu'elle puisse porter le même nom quels que soient les paramètres utilisés. On voudrait créer ces fonctions de la manière suivante :

```
int     Max(int a, int b)           { ... }
float   Max(float a, float b)       { ... }
double  Max(double a, double b)     { ... }
...
k = Max(u,v) ;
```

Le compilateur connaît les types des variables u et v, il va donc chercher la fonction Max prenant ces types là en arguments. Les différentes fonctions Max ne traitent pas les mêmes éléments, pourtant conceptuellement, elles effectuent le même calcul.

POLYMORPHISME D'HÉRITAGE

Le polymorphisme d'héritage va consister à redéclarer dans une classe fille, une méthode déjà déclarée dans une classe parent.

Contrairement au polymorphisme ad hoc qui n'utilise pas de mot clé particulier, on utilisera ici le couple virtual/override.

La première déclaration de la méthode doit porter le qualificatif virtual et toutes les classes enfants modifiant son comportement doivent précéder sa déclaration par le qualificatif override.

```
class A {
        public virtual void Afficher(){
                WriteLine("Bonjour");
        }
}
class B : A {
        public override void Afficher() {
                WriteLine("Coucou");
        }
}
```

LES METHODES D EXTENSION

Une méthode d'extension permet d'étendre une classe en lui rajoutant des méthodes. Ceci est pratique lorsque nous ne possédons pas le code source d'une classe et qu'il s'avère difficile de la modifier. Pour définir et appeler une méthode d'extension en C# il ne faut que respecter les règles suivants :

1. Définir une classe static visible pour le client et comportant la méthode d'extension (p.ex. public).
2. Implémenter la méthode d'extension avec la même visibilité que la classe et le mot clé static.
3. Le premier paramètre de la méthode spécifie le type de donnés qui est élargi. Ce paramètre doit compter le modificateur this.
4. Dans le code qui appelle la méthode il faut inclure le namespace qui comporte la méthode d'extension.
5. Appeler les méthodes comme c'était une méthode instance de ce type de donnés.

LES METHODES D EXTENSION

Créer une méthode d'extension

Nous souhaitons créer une méthode d extension permettant de crypter/decrypter une chaine de caractères.

```
public static class Encodage
{
    public static string Crypte(string chaine)
    {
        return Convert.ToBase64String(Encoding.Default.GetBytes(chaine));
    }

    public static string Decrypte(string chaine)
    {
        return Encoding.Default.GetString(Convert.FromBase64String(chaine));
    }
}
```

LES METHODES D EXTENSION

```
class Program
{
    static void Main(string[] args)
    {
        string chaineNormale = "Bonjour à tous";
        string chaineCryptee = Encodage.Crypte(chaineNormale);
        Console.WriteLine(chaineCryptee);
        chaineNormale = Encodage.Decrypte(chaineCryptee);
        Console.WriteLine(chaineNormale);
    }

}
```

LES METHODES D EXTENSION

Utiliser une méthode d'extension

Il est possible de faire en sorte que ces deux méthodes deviennent des méthodes d'extensions de la classe **string**. Il suffit d'utiliser le mot-clé this devant le premier paramètre de la méthode afin de créer une méthode d'extension

```
public static class Encodage
{
    public static string Crypte(this string chaine)
    {
        return Convert.ToBase64String(Encoding.Default.GetBytes(chaine));
    }

    public static string Decrypte(this string chaine)
    {
        return Encoding.Default.GetString(Convert.FromBase64String(chaine));
    }
}
```

LES METHODES D EXTENSION

Utiliser une méthode d'extension

Nous pourrons désormais utiliser ces méthodes comme si elles faisaient parties de la classe String

```
string chaineNormale = "Bonjour à tous";
string chaineCryptee = chaineNormale.Crypte();
Console.WriteLine(chaineCryptee);
chaineNormale = chaineCryptee.Decrypte();
Console.WriteLine(chaineNormale);
```